97.1
Pass

Harish M

PDF generated at: 15 Feb 2026 13:16:09 UTC

View this report on HackerRank

Score

97.1% • 170.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 45 min 23 sec on 15 Feb 2026 17:58:43 IST

Candidate Information

Email	harisheee2306@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	15 Feb 2026 17:58:43 IST
Time taken	45 min 23 sec/ 90 min
Work Experience	4 years
Invited by	Kalyani
Invited on	13 Feb 2026 19:04:49 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	75%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	75%	Hard	100%
Spring	50%	Application Properties Annotations	0%
Medium	97%	Exception Handling	100%
Aspect-Oriented Programming	100%	Graphs	100%
Greedy	100%	Java	100%

Questions

Spring Boot (Advanced) • 20 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Cracker Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 48 sec	✓

2	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	0/5	-	1 min 33 sec	✗
3	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 21 sec	✓
4	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 48 sec	✓
5	Predict the Output Multiple Choice	Spring Boot (Intermediate)	5/5	-	3 min 35 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Minimize Path Value Coding	Problem Solving (Intermediate)	75/75	-	16 min 49 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Optimized Sequential Video Watching Coding	Java (Intermediate)	75/75	-	16 min 40 sec	✓

1. Cracker

Correct

Multiple Choice | Spring Boot | Hard

Question description

Cracker.java

```
@Component
public class Cracker {
    public void startCracking(){
        System.out.println("Cracking...");
    }
    public void stopCracking(){
        System.out.println("Cracking Stopped!");
    }
}
```

CrackerService.java

```
@Service
public class CrackerService {

    @Autowired
    Cracker cracker;
    public void workWithNewCracker() {
        cracker.stopCracking();
        cracker = new Cracker();
    }
    public void startWorking() {
        cracker.startCracking();
    }
}
```

CrackerWorker.java

```
@Service
public class CrackerWorker {

    @Autowired
    CrackerService crackerService;

    public void startWorking() {
        crackerService.startWorking();
```

```
        crackerService.workWithNewCracker();
        crackerService = new CrackerService();
        crackerService.startWorking();
    }
}
```

Consider these classes are managed in Spring Boot. What happens when 'crackerWorker.startWorking()' is called in the Spring context?

Interviewer guidelines

CrackerService in CrackerWorker will work perfectly until it's replaced with new CrackerService. When that happens it just replace existing injected bean with empty pojo. Which doesn't have it's cracker object so when it's startWorking method is called it will try to execute null cracker attributes startWorking method and NullPointerException will arise.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ "Cracking..." "Cracking Stopped!" will be written to console
- ○ "Cracking..." "Cracking Stopped!" "Cracking..." will be written to console
- ● **"Cracking..." "Cracking Stopped!" NullPointerException will be thrown** ✓
- ○ None of above

No comments.

2. Configuration Properties

Incorrect

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
```

```
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).
- ○ The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- ○ A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final.
- ● ** An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation. **

A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

3. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope(“request”) creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope(“request”).
- [x] @Scope(“prototype”) will return a different instance every time it is requested from the container.

No comments.

4. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- private void manage(HttpSecurity http) {   http.csrf().allowed() }
- private void manage(HttpSecurity http) {   http.csrf() }
- private void manage(HttpSecurity http) {   http.csrf().enable() }

None of the above. CSRF is configured by default when Spring Security Dependency is added.

No comments.

5. Predict the Output

Correct

Multiple Choice | Medium | Exception Handling | Spring Boot | Aspect-Oriented Programming

Question description

Consider the following Snippet and predict the output.

The following API is triggered: http://localhost:8080/add/employee?eId=123&fName=Ravi&lastName=Shekhar

```
@Autowired
private EService eService;

@RequestMapping(value = "/add/employee", method = RequestMethod.GET)
public com.model.Employee add(@RequestParam("eId") String eId, @RequestParam("fName") Strin
g fName, @RequestParam("lastName") String lastName)
{
   return eService.createEmployee(eId, fName, lastName);
}

@Aspect
@Component
public class EServiceAspect
{
   @After(value = "execution(* com.random.service.EService.*(..)) && args(eId, fName, lastName)")
   public void afterAdvice(JoinPoint joinPoint, String eId, String fName, String lastName) {
      System.out.println("After method:" + joinPoint.getSignature());
      System.out.println("Creating Employee with first name - " + fName+ ", second name - " + lastNa
me+ " and id - " + eId);
```

```
    }
  }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(string,string,string) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(empId,firstName,secondName) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Shekhar, second name - Ravi and id - 123

No comments.

6. Minimize Path Value

Correct

Coding Graphs Medium

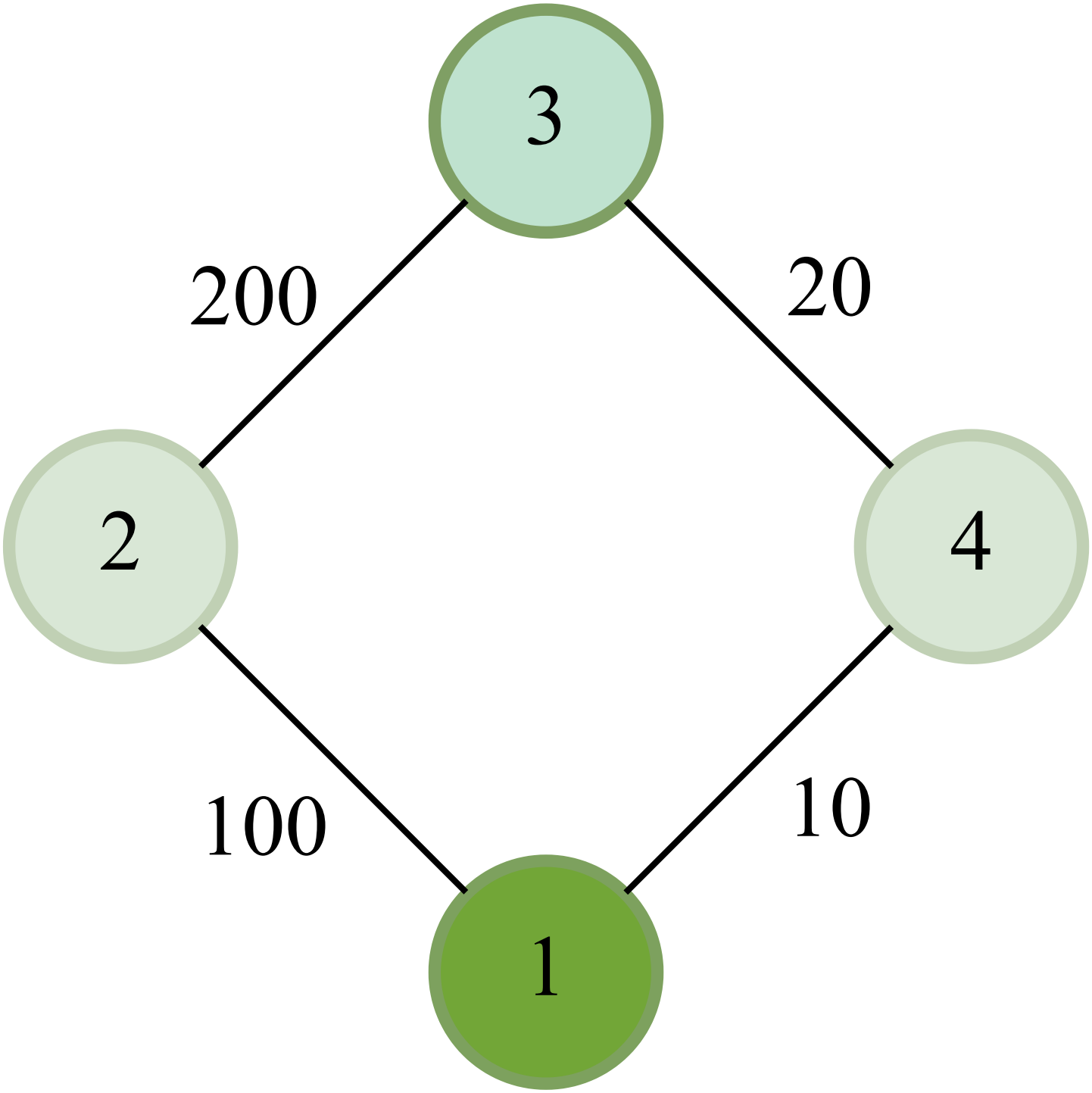
Question description

You have a weighted undirected graph with *graph_nodes* nodes and *m* edges. The stress level of a path between two nodes is defined as the weight of the heaviest edge in that path.

Given a source node *source* and a destination node *destination*, find the minimum possible stress level of a path. If no such path exists, return -1.

Example

Consider the following example below, where *source=1* and *destination=3.*



There are two paths, from node 1 to node 3.

1. The max weighted edge/stress level in path 1 -> 2 -> 3 is 200.
2. The max weighted edge/stress level in the path 1 -> 4 -> 3 is 20.

Return 20, the lower stress level from the second path.

Function Description

Complete the function *getMinimumStress* in the editor with the following parameter(s):

int graph_nodes: number of nodes
int graph_from[m]: one end node for an edge
int graph_to[m]: one end node for an edge
int graph_weight[m]: the weights of the edges
int source: the source node
int destination: the destination node

Returns

int: the value of the least stressful path, or -1 if no path exists

Note: Even though edge endpoints are called *graph_from[i]* and *graph_to[i]*, this is an undirected graph.

Constraints

- $1 \le graph_nodes \le 10^5$
- $1 \le |graph_from| = |graph_to| = |graph_weight| \le 10^5$ *(Where |A| denotes the size of array A)*
- $1 \le graph_from[i], graph_to[i], source, destination \le graph_nodes$
- $0 \le graph_weight[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains two space-separated integers *graph_nodes* and *m,* the number of nodes, and the number of edges of the graph.
Each of the following *m* lines contains three space-separated integers *u, v,* and *w,* meaning there is an edge connecting the nodes *u* and *v* of weight *w.*
The next line contains a single integer, denoting the index of the "*source*" node.
The next line contains a single integer, denoting the index of the "*destination*" node.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

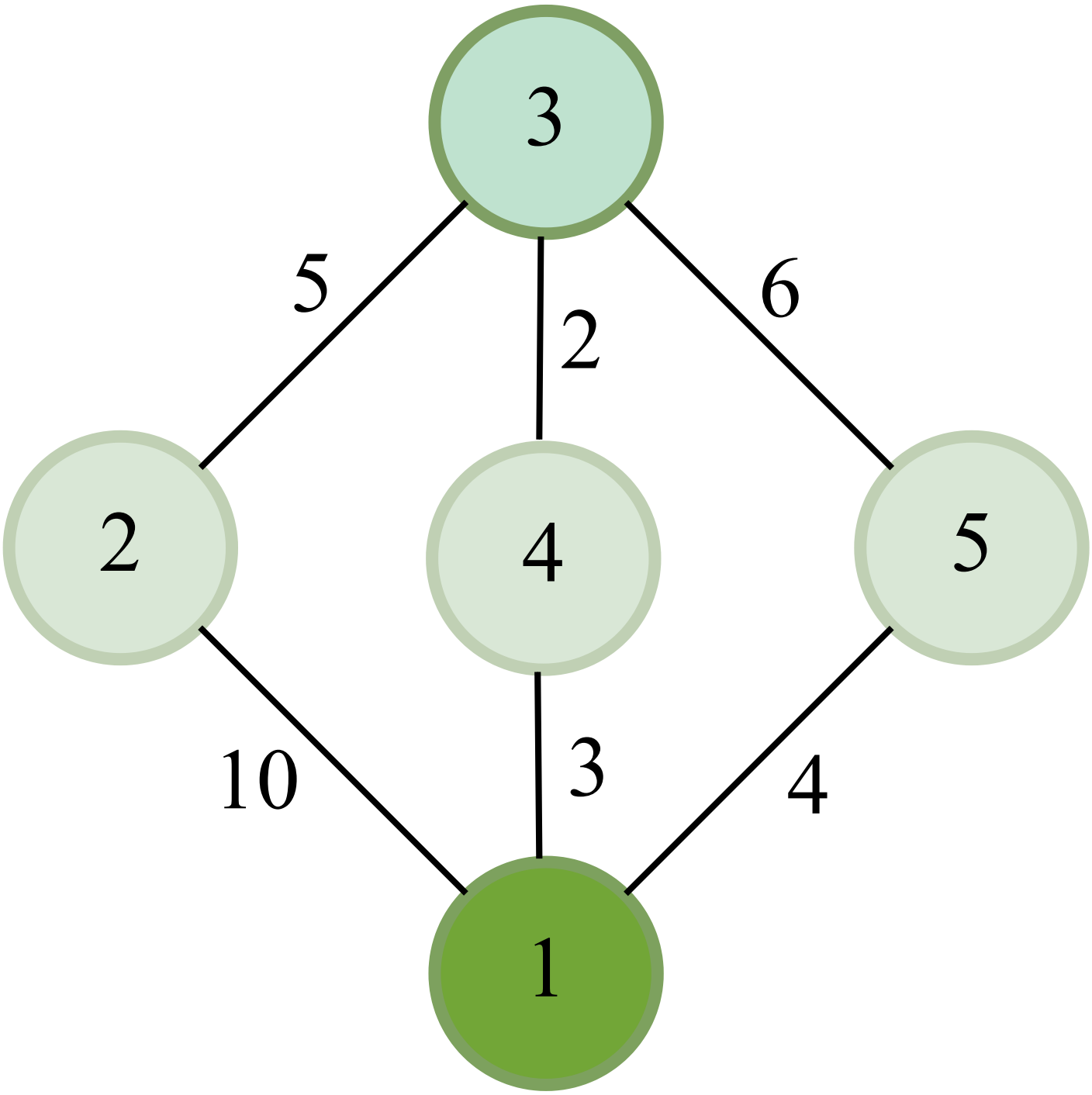
```
STDIN       FUNCTION
-----       --------
5 6         graph_nodes = 5, m = 6
1 2 10      u = 1, v = 2, w = 10
```

```
2 3 5
1 4 3
4 3 2
1 5 4
5 3 6
1          source = 1
3          destination = 3
```

Sample Output

```
3
```

Explanation



There are three paths, from node 1 to node 3 :

1. 1 -> 2 -> 3. Edges have weights 10 and 5, the maximum weighted edge, and the stress level is 10.
2. 1 -> 4 -> 3. Edges have weights 3 and 2, and the stress level = 3.
3. 1 -> 5 -> 3. Edges have weights 4 and 6, and the stress level = 6.

▼ SAMPLE CASE 1

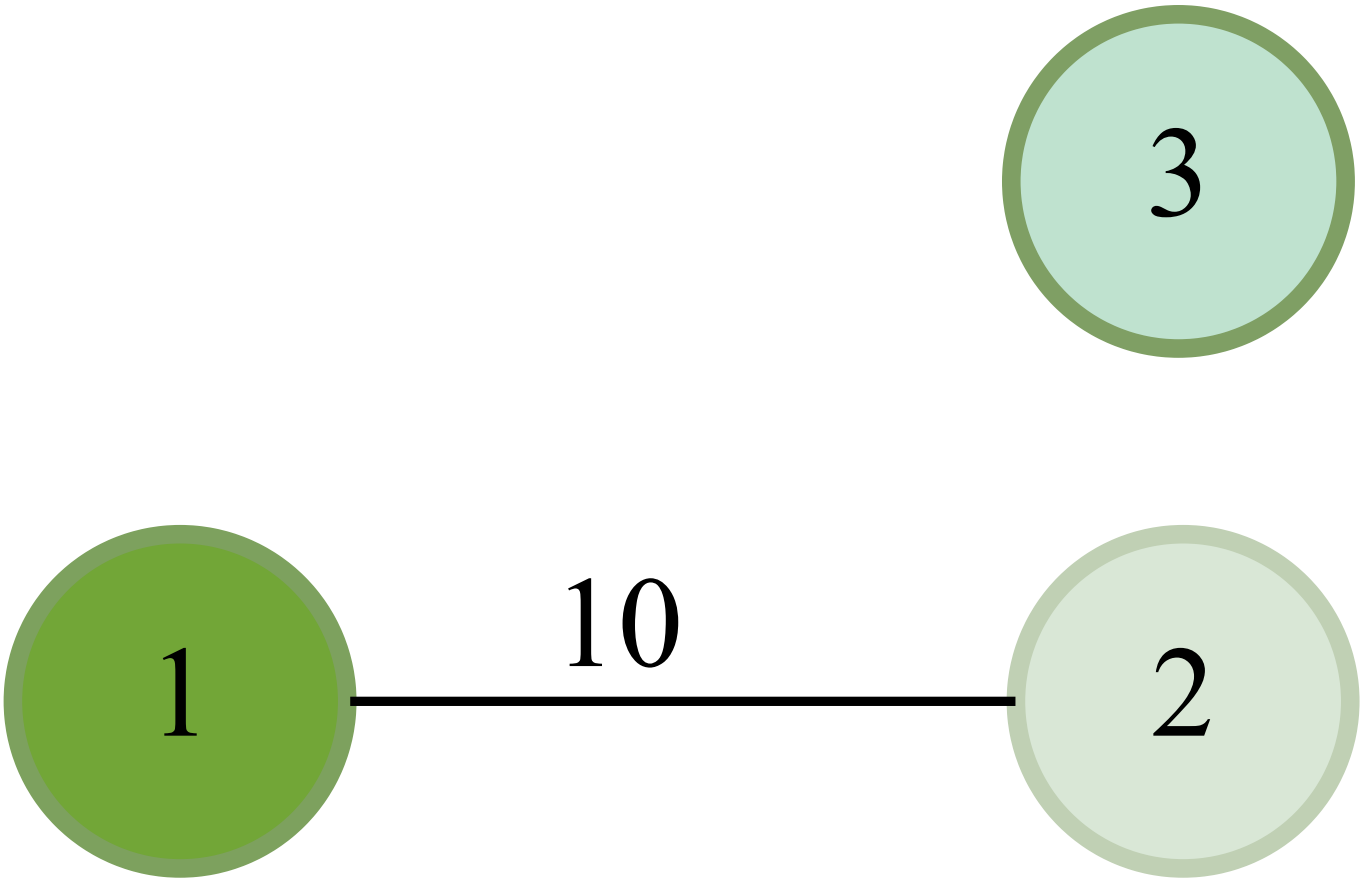
Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3 1          graph_nodes = 3, m = 1
1 2 10       a = 1, b = 2, w = 10
1           source = 1
3           destination = 3
```

Sample Output

```
-1
```

Explanation



There is no path, from node 1 to node 3.

Interviewer guidelines

▼ SOLUTION

Python Solution

```
sys.setrecursionlimit(10**5)


def getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination):

    adj = [[] for i in range(graph_nodes + 1)]
    vis = [False] * (graph_nodes + 1)

    for x, y, w in zip(graph_from, graph_to, graph_weight):
        adj[x].append((y, w))
        adj[y].append((x, w))

    def dfs(node, mx):
        vis[node] = True
        if node == destination:
            return True
        ans = False
        for i, w in adj[node]:
            if w <= mx and not vis[i]:
                ans |= dfs(i, mx)
        return ans

    if not dfs(source, max(graph_weight)):
        return -1

    lo, hi = 0, max(graph_weight)
    while hi - lo > 1:
        vis = [False] * (graph_nodes + 1)
        mid = (hi + lo) // 2
        if dfs(source, mid):
            hi = mid
        else:
            lo = mid
    return hi
```

Candidate's Solution

Language used: **Java 17**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
```

```java
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMinimumStress' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. WEIGHTED_INTEGER_GRAPH graph
     *  2. INTEGER source
     *  3. INTEGER destination
     */

    /*
     * For the weighted graph, <name>:
     *
     * 1. The number of nodes is <name>Nodes.
     * 2. The number of edges is <name>Edges.
     * 3. An edge exists between <name>From[i] and <name>To[i]. The weight of the edge is <name>Weight[i].
     *
     */

    static class Edge implements Comparable<Edge>{

        int u, v, weight;

        Edge(int u, int v, int weight){
            this.u = u;
            this.v = v;
            this.weight = weight;
        }

        @Override
        public int compareTo(Edge o){
            return Integer.compare(this.weight, o.weight);
        }
    }

```

```java
    public static int getMinimumStress(int graphNodes, List<Integer> graphFrom, List<Integer> graphTo, List<Integer> graphWeight, int source, int destination) {

        List<Edge> list = new ArrayList<>();

        int m = graphFrom.size();

        for(int i = 0; i< m; i++){

            list.add(new Edge(graphFrom.get(i), graphTo.get(i), graphWeight.get(i)));
        }

        Collections.sort(list);

        DSU dsu = new DSU(graphNodes);

        for(Edge edge: list){
            dsu.union(edge.u, edge.v);

            if(dsu.find(source) == dsu.find(destination)){
                return edge.weight;
            }
        }

        return -1;



    }


    static class DSU{
        int[] parent;

        DSU(int n){
            parent = new int[n+1];
            for(int i =0;i <= n; i++){
                parent[i] = i;
            }
        }

        public int find(int i){
            if(parent[i] ==i) return i;
            return parent[i] = find(parent[i]);
```

```
        }

        public void union(int i, int j){
            int rootI = find(i);
            int rootJ = find(j);
            if(rootI != rootJ){
                parent[rootI] = rootJ;
            }
        }
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        String[] graphNodesEdges = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int graphNodes = Integer.parseInt(graphNodesEdges[0]);
        int graphEdges = Integer.parseInt(graphNodesEdges[1]);

        List<Integer> graphFrom = new ArrayList<>();
        List<Integer> graphTo = new ArrayList<>();
        List<Integer> graphWeight = new ArrayList<>();

        IntStream.range(0, graphEdges).forEach(i -> {
            try {
                String[] graphFromToWeight = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

                graphFrom.add(Integer.parseInt(graphFromToWeight[0]));
                graphTo.add(Integer.parseInt(graphFromToWeight[1]));
                graphWeight.add(Integer.parseInt(graphFromToWeight[2]));
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        int source = Integer.parseInt(bufferedReader.readLine().trim());

```

```
        int destination = Integer.parseInt(bufferedReader.readLine().trim());

        int result = Result.getMinimumStress(graphNodes, graphFrom, graphTo, graphWeight, source, destination);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0488 sec	33.5 KB
TestCase 1	Easy	Sample	Success	1	0.0457 sec	33.7 KB
TestCase 2	Easy	Sample	Success	1	0.0486 sec	33.6 KB
TestCase 3	Easy	Hidden	Success	6	0.5162 sec	90.8 KB
TestCase 4	Easy	Hidden	Success	6	0.4236 sec	84.4 KB
TestCase 5	Easy	Hidden	Success	6	0.5178 sec	84.8 KB
TestCase 6	Easy	Hidden	Success	6	0.6347 sec	98.8 KB
TestCase 7	Easy	Hidden	Success	6	0.5804 sec	102 KB

TestCase 8	Easy	Hidden	Success	6	0.5313 sec	93.6 KB
TestCase 9	Easy	Hidden	Success	6	0.5913 sec	93.6 KB
TestCase 10	Easy	Hidden	Success	6	0.5911 sec	101 KB
TestCase 11	Easy	Hidden	Success	6	0.567 sec	93.7 KB
TestCase 12	Easy	Hidden	Success	6	0.2592 sec	64.5 KB
TestCase 13	Easy	Hidden	Success	6	0.4085 sec	73.4 KB
TestCase 14	Easy	Hidden	Success	6	0.5757 sec	100 KB

No comments.

7. Java: Optimized Sequential Video Watching

Correct

Coding Medium Greedy Java

Question description

A learner is given a sequential playlist of *n* videos to enhance their knowledge. Implement a function to calculate the minimum time required to complete a total of *m* viewings while adhering to the platform's constraints.

System Constraints

- Sequential Viewing:
  - The learner must watch videos in the order they appear in the playlist.
  - To watch a video at index *i*, the learner must first watch all videos from index 0 to *i-1* at least once.
- Initial Full Viewing:
  - To view any video at index *i* for the first time, the learner must spend *firstWatch[i] + repeatWatch[i]* minutes. This ensures a complete understanding of the video.
- Subsequent Viewings:
  - For subsequent viewings of a video at index *i*, the learner spends only *repeatWatch[i]* minutes.
  - At any time, they can rewatch any video they have already watched.
- Minimum Total Watch Count:
  - The learner must achieve at least *m* total viewings across all videos.
  - Videos can be rewatched as desired, and not all videos need to be watched equally.

Return the minimum time required to complete *m* viewings of videos while following these constraints.

Example

If *n* = 4, *m* = 4, *firstWatch* = [1, 5, 9, 11], and *repeatWatch* = [2, 7, 10, 11], the optimal strategy is as follows:

- The learner watches the 0th video for the first time, spending *firstWatch*[0] + *repeatWatch*[0] = 1 + 2 = 3 minutes.
- The learner then rewatches the 0th video 3 more times in, spending *repeatWatch*[0]*3 = 2 * 3 = 6 minutes.
- The learner has successfully completed 4 viewings. Thus, the total time required is 3 + 6 = 9 minutes, and 9 is returned.

Function Description

Complete the function *findMinMinutes* in the editor below.

Function Parameters

int m: the total number of video viewings to complete

int firstWatch[n]: the minutes required to watch each video the first time

int repeatWatch[n]: the minutes required to rewatch each video

Returns

long: the minimum minutes required for the learner to reach their target number of viewings

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^9$
- $1 \le firstWatch[i], repeatWatch[i] \le 10^9$

INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *m*.
The next line contains an integer, *n,* the size of *firstWatch*.
Each of the next *n* lines contains an integer, *firstWatch[i]*.
The next line contains an integer, *n,* the size of *repeatWatch*.
Each of the next *n* lines contains an integer, *repeatWatch[i]*.

SAMPLE CASE 0

Sample Input 0

```
STDIN    FUNCTION
-----    --------
5        m = 5
3        firstWatch[] size n = 3
6        firstWatch = [6, 8, 9]
8
9
3        repeatWatch[] size n = 3
4        repeatWatch = [4, 2, 10]
2
10
```

Sample Output 0

```
26
```

Explanation

The goal is to complete 5 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time, so the time required is *firstWatch*[0] + *repeatWatch*[0] = 6 + 4 = 10 minutes.
- The learner watches the 1st video for the first time, so the time required is *firstWatch*[1] + *repeatWatch*[1] = 8 + 2 = 10 minutes.
- The learner then rewatches the 1st video 3 times. So the time taken = *repeatWatch*[1]*3 = 2 * 3 = 6 minutes.

- The total time required to complete 5 video viewings is 10 + 10 + 6 = 26 minutes.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
6         m = 6
5         firstWatch[] size n = 5
2         firstWatch = [2, 3, 7, 8, 4]
3
7
8
4
5         repeatWatch[] size n = 5
3         repeatWatch = [3, 6, 8, 7, 6]
6
8
7
6
```

Sample Output 1

```
20
```

Explanation

The goal is to complete 6 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time in *firstWatch*[0] + *repeatWatch[0]* = 2 + 3 = 5 minutes.
- The learner then rewatches the 0th video 5 times in *repeatWatch[0]**5 = 3 * 5 = 15 minutes.
- The total time required to complete 6 viewings is 5 + 15 = 20 minutes.

Interviewer guidelines

▼ SOLUTION

Description

The greedy solution suggests that, at any given index, if the time to watch the video is less than at any other index between i+1 and n, the minimum total time will be the sum of the watch time up to index i-1, plus the repeat time of the ith video multiplied by (m-i).

Solution

```
class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
        long minMinutes = Long.MAX_VALUE;
        long cur = 0;
        int n = firstWatch.size();

        for (int i = 0; i < Math.min(n,m); i++) {
            int a = firstWatch.get(i);
            int b = repeatWatch.get(i);

            minMinutes = Math.min(minMinutes, cur + ((long) (m - i) * b) + a);

            cur += a + b;
        }
        return minMinutes;
    }

}
```

▼ COMPLEXITY

Time Complexity - O(n)

The algorithm iterates through each number in *firstWatch* and *repeatWatch* exactly once, performing constant-time checks and arithmetic operations in each iteration. Thus, the time complexity is O(n), where *n* is the length of *firstWatch*.

Space Complexity - O(1)

Space usage is constant, only utilizing a fixed number of variables (*mn, cur,* etc.) that do not scale with the length of *firstWatch*. Therefore, the space complexity is O(1).

Candidate's Solution

Language used: **Java 17**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {

        long minTotalTime = Long.MAX_VALUE;
        long current = 0;
        long repeatSoFar = Long.MAX_VALUE;

        int size = firstWatch.size();

        for(int i = 0; i < size; i++){

            long currentCost = (long)firstWatch.get(i) + repeatWatch.get(i);
            current += currentCost;

            int currentRepeat = repeatWatch.get(i);
            if(currentRepeat < repeatSoFar){
                repeatSoFar = currentRepeat;
            }

```

```
            long viewsCompleted = i+1;
            long viewsRemaining = m - viewsCompleted;

            long totalTime;

            if(viewsRemaining <= 0){
                totalTime = current;
            }else{
                totalTime = current + (viewsRemaining * repeatSoFar);
            }

            if(totalTime < minTotalTime){
                minTotalTime = totalTime;
            }

        }

        return minTotalTime;

    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int m = Integer.parseInt(bufferedReader.readLine().trim());

        int firstWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> firstWatch = IntStream.range(0, firstWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

```

```
        int repeatWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> repeatWatch = IntStream.range(0, repeatWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.findMinMinutes(m, firstWatch, repeatWatch);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 2	Easy	Sample	Success	1	0.0478 sec	33.7 KB
Testcase 0	Easy	Sample	Success	1	0.0552 sec	33.8 KB
Testcase 1	Easy	Sample	Success	1	0.0442 sec	33.6 KB
Testcase 3	Easy	Hidden	Success	4	0.0512 sec	33.6 KB
Testcase 4	Easy	Hidden	Success	4	0.062 sec	34.4 KB

Testcase 5	Easy	Hidden	Success	4	0.0911 sec	41.7 KB
Testcase 6	Medium	Hidden	Success	5	0.2956 sec	62.8 KB
Testcase 7	Medium	Hidden	Success	5	0.5155 sec	163 KB
Testcase 8	Medium	Hidden	Success	5	0.485 sec	163 KB
Testcase 9	Medium	Hidden	Success	5	0.5506 sec	162 KB
Testcase 10	Medium	Hidden	Success	5	0.5761 sec	166 KB
Testcase 11	Medium	Hidden	Success	5	0.4094 sec	128 KB
Testcase 12	Hard	Hidden	Success	10	0.2456 sec	62.8 KB
Testcase 13	Hard	Hidden	Success	10	0.4891 sec	112 KB
Testcase 14	Hard	Hidden	Success	10	0.0643 sec	33.6 KB

No comments.